

KLASIFIKASI JENIS BERAS BERDASARKAN CITRA DIGITAL

Nama: Sakti Attila Aulia Bintang

NIM: 41524010126

Kelas: Pengolahan citra

Tujuan

Melakukan klasifikasi jenis beras berdasarkan citra digital menggunakan tahapan pengolahan citra, mulai dari akuisisi citra, pre-processing, enhancement dan restoration, segmentasi, ekstraksi fitur, hingga klasifikasi.

Jenis beras yang digunakan terdiri dari:

1. Beras Putih
2. Beras Merah
3. Beras Hitam

1. Akuisisi Citra

Akuisisi citra merupakan tahap awal dalam pengolahan citra yang bertujuan untuk memperoleh citra digital yang akan digunakan sebagai data masukan.

Pada penelitian ini digunakan tiga citra sebagai data masukan, yaitu citra beras putih, beras merah, dan beras hitam. Citra diperoleh dalam format JPEG dan selanjutnya dibaca menggunakan library OpenCV.

```
# IMPORT LIBRARY

import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from google.colab import files
from google.colab.patches import cv2_imshow
```

Upload Citra

Tiga citra jenis beras diunggah ke Google Colab sebagai data masukan. Citra yang digunakan terdiri dari beras putih, beras merah, dan beras hitam.

```
uploaded = files.upload()
```

Choose Files 3 files

Beras Putih.jpeg(image/jpeg) - 71544 bytes, last modified: 9/21/2026 - 100% done

Beras Merah.jpeg(image/jpeg) - 41190 bytes, last modified: 9/21/2026 - 100% done

Beras Hitam.jpeg(image/jpeg) - 157629 bytes, last modified: 9/21/2026 - 100% done

Saving Beras Putih.jpeg to Beras Putih (2).jpeg

Saving Beras Merah.jpeg to Beras Merah (2).jpeg

Saving Beras Hitam.jpeg to Beras Hitam (2).jpeg

✓ Membaca Citra

Setelah citra berhasil diunggah, masing-masing citra dibaca menggunakan fungsi `cv2.imread()` untuk diproses pada tahap berikutnya.

```
# MEMBACA CITRA
```

```
img_putih = cv2.imread("Beras Putih.jpeg")
```

```
img_merah = cv2.imread("Beras Merah.jpeg")
```

```
img_hitam = cv2.imread("Beras Hitam.jpeg")
```

```
print("Citra berhasil dibaca.")
```

```
print("Beras Putih :", img_putih.shape)
```

```
print("Beras Merah :", img_merah.shape)
```

```
print("Beras Hitam :", img_hitam.shape)
```

```
Citra berhasil dibaca.
```

```
Beras Putih : (449, 446, 3)
```

```
Beras Merah : (283, 325, 3)
```

```
Beras Hitam : (628, 619, 3)
```

✓ Visualisasi Citra Hasil Akuisisi

Citra hasil akuisisi ditampilkan untuk memastikan bahwa seluruh citra berhasil dibaca dengan baik dan dapat digunakan sebagai data masukan untuk tahap pengolahan selanjutnya.

```
# VISUALISASI CITRA HASIL AKUISISI
```

```
plt.figure(figsize=(15, 5))
```

```
# Beras Putih
```

```
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img_putih, cv2.COLOR_BGR2RGB))
plt.title("Beras Putih")
plt.axis("off")

# Beras Merah
plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(img_merah, cv2.COLOR_BGR2RGB))
plt.title("Beras Merah")
plt.axis("off")

# Beras Hitam
plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(img_hitam, cv2.COLOR_BGR2RGB))
plt.title("Beras Hitam")
plt.axis("off")

plt.tight_layout()
plt.show()
```



✓ 2. Pre-processing

Pre-processing merupakan tahap awal pengolahan citra yang dilakukan untuk mempersiapkan citra sebelum masuk ke tahap pengolahan berikutnya.

Pada tahap ini dilakukan beberapa proses, yaitu:

1. **Resize** untuk menyeragamkan ukuran citra.
2. **Grayscale** untuk memperoleh citra dalam tingkat keabuan.
3. **Noise Reduction** menggunakan Gaussian Blur untuk mengurangi noise pada citra.

Informasi warna dari citra asli tetap dipertahankan karena akan digunakan pada tahap ekstraksi fitur dan klasifikasi.

✓ 2.1 Resize Citra

Resize dilakukan untuk menyeragamkan ukuran ketiga citra sehingga citra memiliki dimensi yang sama dan lebih mudah diproses pada tahap selanjutnya.

```
# RESIZE CITRA

# Ukuran standar: lebar 600 piksel, tinggi 400 piksel
target_size = (600, 400)

putih_resize = cv2.resize(img_putih, target_size)
merah_resize = cv2.resize(img_merah, target_size)
hitam_resize = cv2.resize(img_hitam, target_size)

print("Resize citra selesai.")
print("Ukuran Beras Putih :", putih_resize.shape)
print("Ukuran Beras Merah :", merah_resize.shape)
print("Ukuran Beras Hitam :", hitam_resize.shape)
```

```
Resize citra selesai.
Ukuran Beras Putih : (400, 600, 3)
Ukuran Beras Merah : (400, 600, 3)
Ukuran Beras Hitam : (400, 600, 3)
```

✓ Visualisasi Hasil Resize

Berikut merupakan hasil citra setelah dilakukan proses resize. Ketiga citra telah diseragamkan menjadi ukuran 600 × 400 piksel.

VISUALISASI HASIL RESIZE

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(putih_resize, cv2.COLOR_BGR2RGB))
plt.title("Beras Putih - Resize")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(merah_resize, cv2.COLOR_BGR2RGB))
plt.title("Beras Merah - Resize")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(hitam_resize, cv2.COLOR_BGR2RGB))
plt.title("Beras Hitam - Resize")
plt.axis("off")

plt.tight_layout()
plt.show()
```



✓ 2.2 Konversi Grayscale

Konversi grayscale dilakukan untuk memperoleh representasi citra dalam tingkat keabuan. Citra grayscale dapat digunakan untuk membantu proses pengolahan berdasarkan intensitas dan bentuk objek.

Citra warna asli tetap disimpan dan tidak diubah karena informasi warna diperlukan pada tahap ekstraksi fitur dan klasifikasi.

KONVERSI KE GRAYSCALE

```
putih_gray = cv2.cvtColor(putih_resize, cv2.COLOR_BGR2GRAY)
merah_gray = cv2.cvtColor(merah_resize, cv2.COLOR_BGR2GRAY)
hitam_gray = cv2.cvtColor(hitam_resize, cv2.COLOR_BGR2GRAY)

print("Konversi grayscale selesai.")
```

Konversi grayscale selesai.

✓ Visualisasi Citra Grayscale

Citra hasil konversi grayscale ditampilkan untuk melihat perubahan citra dari format berwarna menjadi tingkat keabuan.

```
# VISUALISASI GRAYSCALE

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(putih_gray, cmap="gray")
plt.title("Beras Putih - Grayscale")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(merah_gray, cmap="gray")
plt.title("Beras Merah - Grayscale")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(hitam_gray, cmap="gray")
plt.title("Beras Hitam - Grayscale")
plt.axis("off")

plt.tight_layout()
plt.show()
```



✓ 2.3 Noise Reduction

Noise reduction dilakukan untuk mengurangi gangguan atau noise pada citra. Pada penelitian ini digunakan metode **Gaussian Blur** dengan kernel 5×5 .

Proses ini dilakukan pada citra grayscale sebagai persiapan untuk tahap pengolahan citra berikutnya.

```
# NOISE REDUCTION - GAUSSIAN BLUR
```

```
putih_pre = cv2.GaussianBlur(putih_gray, (5, 5), 0)
merah_pre = cv2.GaussianBlur(merah_gray, (5, 5), 0)
hitam_pre = cv2.GaussianBlur(hitam_gray, (5, 5), 0)

print("Noise reduction dengan Gaussian Blur selesai.")
```

Noise reduction dengan Gaussian Blur selesai.

✓ Visualisasi Hasil Pre-processing

Hasil akhir pre-processing ditampilkan untuk membandingkan citra grayscale sebelum dan sesudah dilakukan noise reduction.

```
# PERBANDINGAN HASIL PRE-PROCESSING

plt.figure(figsize=(15, 10))

# Beras Putih
plt.subplot(3, 2, 1)
plt.imshow(putih_gray, cmap="gray")
plt.title("Beras Putih - Grayscale")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(putih_pre, cmap="gray")
plt.title("Beras Putih - Setelah Noise Reduction")
plt.axis("off")

# Beras Merah
plt.subplot(3, 2, 3)
plt.imshow(merah_gray, cmap="gray")
plt.title("Beras Merah - Grayscale")
plt.axis("off")

plt.subplot(3, 2, 4)
plt.imshow(merah_pre, cmap="gray")
plt.title("Beras Merah - Setelah Noise Reduction")
plt.axis("off")

# Beras Hitam
plt.subplot(3, 2, 5)
plt.imshow(hitam_gray, cmap="gray")
plt.title("Beras Hitam - Grayscale")
plt.axis("off")

plt.subplot(3, 2, 6)
plt.imshow(hitam_pre, cmap="gray")
plt.title("Beras Hitam - Setelah Noise Reduction")
plt.axis("off")
```

```
plt.tight_layout()  
plt.show()
```

Beras Putih - Grayscale



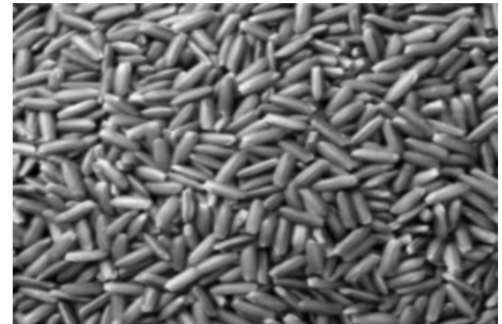
Beras Putih - Setelah Noise Reduction



Beras Merah - Grayscale



Beras Merah - Setelah Noise Reduction



Beras Hitam - Grayscale



Beras Hitam - Setelah Noise Reduction



✓ 3. Enhancement & Restoration

Enhancement dan restoration merupakan tahap untuk meningkatkan kualitas citra sehingga informasi atau detail pada citra dapat terlihat dengan lebih jelas.

Pada tahap ini dilakukan dua proses, yaitu:

1. **Contrast Enhancement** menggunakan metode CLAHE (Contrast Limited Adaptive Histogram Equalization) untuk meningkatkan kontras citra secara lokal.
2. **Sharpening** untuk memperjelas detail dan tepi objek pada citra.

Proses enhancement dilakukan pada citra hasil pre-processing.

✓ 3.1 Contrast Enhancement

Contrast enhancement dilakukan untuk meningkatkan perbedaan intensitas pada citra sehingga objek beras dapat terlihat lebih jelas terhadap area di sekitarnya.

Metode yang digunakan adalah **CLAHE (Contrast Limited Adaptive Histogram Equalization)**.

```
# CONTRAST ENHANCEMENT MENGGUNAKAN CLAHE

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

putih_enhance = clahe.apply(putih_pre)
merah_enhance = clahe.apply(merah_pre)
hitam_enhance = clahe.apply(hitam_pre)

print("Contrast enhancement dengan CLAHE selesai.")
```

Contrast enhancement dengan CLAHE selesai.

✓ Visualisasi Hasil Contrast Enhancement

Berikut merupakan perbandingan citra sebelum dan sesudah dilakukan peningkatan kontras menggunakan CLAHE.

```
# VISUALISASI CONTRAST ENHANCEMENT

plt.figure(figsize=(15, 10))

# Beras Putih
plt.subplot(3, 2, 1)
plt.imshow(putih_pre, cmap="gray")
plt.title("Beras Putih - Sebelum Enhancement")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(putih_enhance, cmap="gray")
plt.title("Beras Putih - Setelah CLAHE")
plt.axis("off")

# Beras Merah
plt.subplot(3, 2, 3)
plt.imshow(merah_pre, cmap="gray")
plt.title("Beras Merah - Sebelum Enhancement")
plt.axis("off")
```

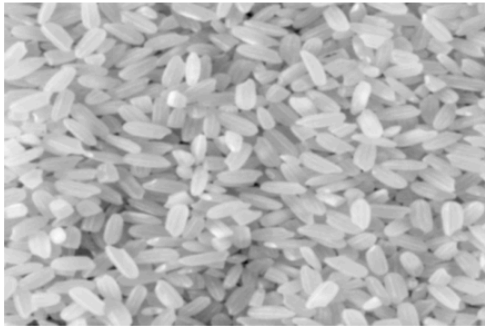
```
plt.subplot(3, 2, 4)
plt.imshow(merah_enhance, cmap="gray")
plt.title("Beras Merah - Setelah CLAHE")
plt.axis("off")

# Beras Hitam
plt.subplot(3, 2, 5)
plt.imshow(hitam_pre, cmap="gray")
plt.title("Beras Hitam - Sebelum Enhancement")
plt.axis("off")

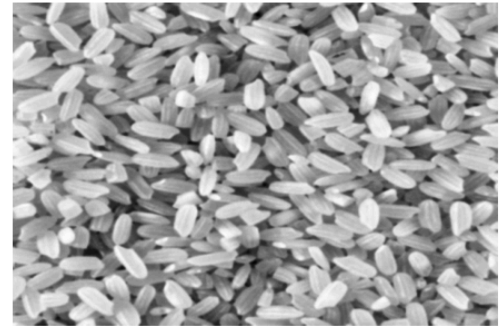
plt.subplot(3, 2, 6)
plt.imshow(hitam_enhance, cmap="gray")
plt.title("Beras Hitam - Setelah CLAHE")
plt.axis("off")

plt.tight_layout()
plt.show()
```

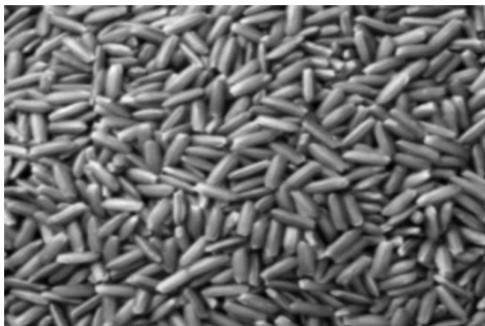
Beras Putih - Sebelum Enhancement



Beras Putih - Setelah CLAHE



Beras Merah - Sebelum Enhancement



Beras Merah - Setelah CLAHE



Beras Hitam - Sebelum Enhancement



Beras Hitam - Setelah CLAHE



3.2 Image Sharpening

Sharpening dilakukan untuk meningkatkan ketajaman citra dengan memperjelas detail dan tepi objek.

Pada tahap ini digunakan **Unsharp Masking**, yaitu dengan menggabungkan citra asli dengan citra hasil Gaussian Blur.

```
# IMAGE SHARPENING - UNSHARP MASKING

def unsharp_mask(image):
    blurred = cv2.GaussianBlur(image, (5, 5), 1.0)
    sharpened = cv2.addWeighted(
        image, 1.5,
        blurred, -0.5,
        0
    )
    return sharpened

putih_sharp = unsharp_mask(putih_enhance)
merah_sharp = unsharp_mask(merah_enhance)
hitam_sharp = unsharp_mask(hitam_enhance)

print("Sharpening selesai.")
```

Sharpening selesai.

✓ Visualisasi Hasil Sharpening

Citra hasil sharpening ditampilkan untuk melihat peningkatan ketajaman detail objek setelah proses enhancement.

```
# VISUALISASI HASIL SHARPENING

plt.figure(figsize=(15, 10))

# Beras Putih
plt.subplot(3, 2, 1)
plt.imshow(putih_enhance, cmap="gray")
plt.title("Beras Putih - Setelah CLAHE")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(putih_sharp, cmap="gray")
plt.title("Beras Putih - Setelah Sharpening")
plt.axis("off")

# Beras Merah
plt.subplot(3, 2, 3)
```

```
plt.imshow(merah_enhance, cmap="gray")
plt.title("Beras Merah - Setelah CLAHE")
plt.axis("off")

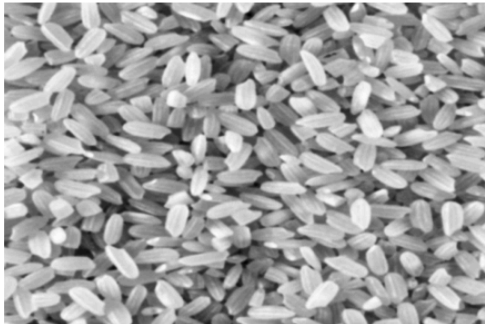
plt.subplot(3, 2, 4)
plt.imshow(merah_sharp, cmap="gray")
plt.title("Beras Merah - Setelah Sharpening")
plt.axis("off")

# Beras Hitam
plt.subplot(3, 2, 5)
plt.imshow(hitam_enhance, cmap="gray")
plt.title("Beras Hitam - Setelah CLAHE")
plt.axis("off")

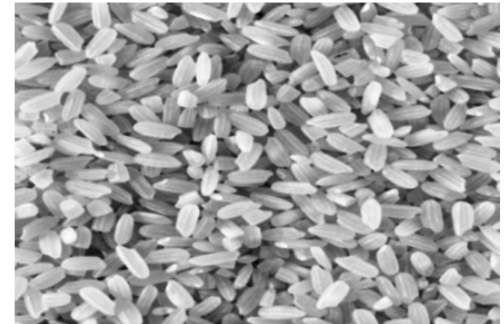
plt.subplot(3, 2, 6)
plt.imshow(hitam_sharp, cmap="gray")
plt.title("Beras Hitam - Setelah Sharpening")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Beras Putih - Setelah CLAHE



Beras Putih - Setelah Sharpening



Beras Merah - Setelah CLAHE



Beras Merah - Setelah Sharpening



Beras Hitam - Setelah CLAHE



Beras Hitam - Setelah Sharpening



Hasil Enhancement & Restoration

Setelah proses enhancement, citra memiliki kontras dan ketajaman yang lebih baik. Citra hasil sharpening selanjutnya digunakan sebagai salah satu masukan untuk proses segmentasi.

✓ 4. Segmentasi

Segmentasi merupakan proses untuk memisahkan objek yang menjadi fokus penelitian dari latar belakang citra.

Pada penelitian ini, objek yang akan dipisahkan adalah butiran beras, sedangkan area selain beras dianggap sebagai background.

Proses segmentasi dilakukan menggunakan thresholding untuk menghasilkan citra biner. Selanjutnya dilakukan operasi morfologi untuk mengurangi noise dan memperbaiki bentuk objek hasil segmentasi.

✓ 4.1 Thresholding

Thresholding digunakan untuk mengubah citra grayscale menjadi citra biner yang terdiri dari dua nilai, yaitu hitam dan putih.

Pada tahap ini digunakan metode **Otsu Thresholding** yang secara otomatis menentukan nilai threshold berdasarkan distribusi intensitas citra.

```
# THRESHOLDING MENGGUNAKAN OTSU

_, putih_thresh = cv2.threshold(
    putih_sharp, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

_, merah_thresh = cv2.threshold(
    merah_sharp, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

_, hitam_thresh = cv2.threshold(
    hitam_sharp, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Thresholding Otsu selesai.")
```

Thresholding Otsu selesai.

✓ Visualisasi Hasil Thresholding

Hasil thresholding ditampilkan dalam bentuk citra biner. Area yang memiliki intensitas tertentu akan dipisahkan menjadi objek foreground dan background.

```
# VISUALISASI HASIL THRESHOLDING

plt.figure(figsize=(15, 10))

plt.subplot(3, 2, 1)
plt.imshow(putih_sharp, cmap="gray")
plt.title("Beras Putih - Sebelum Thresholding")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(putih_thresh, cmap="gray")
plt.title("Beras Putih - Hasil Otsu")
plt.axis("off")

plt.subplot(3, 2, 3)
plt.imshow(merah_sharp, cmap="gray")
plt.title("Beras Merah - Sebelum Thresholding")
plt.axis("off")

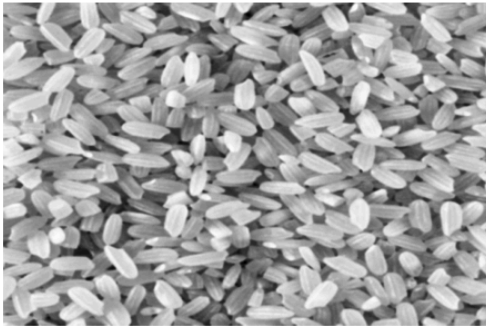
plt.subplot(3, 2, 4)
plt.imshow(merah_thresh, cmap="gray")
plt.title("Beras Merah - Hasil Otsu")
plt.axis("off")
```

```
plt.subplot(3, 2, 5)
plt.imshow(hitam_sharp, cmap="gray")
plt.title("Beras Hitam - Sebelum Thresholding")
plt.axis("off")

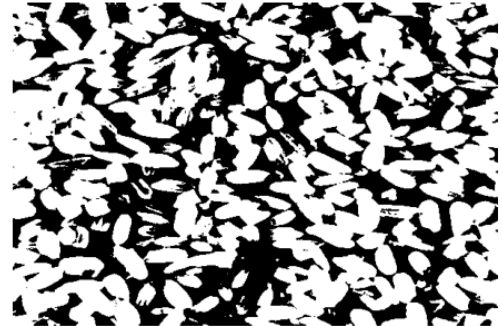
plt.subplot(3, 2, 6)
plt.imshow(hitam_thresh, cmap="gray")
plt.title("Beras Hitam - Hasil Otsu")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Beras Putih - Sebelum Thresholding



Beras Putih - Hasil Otsu



Beras Merah - Sebelum Thresholding



Beras Merah - Hasil Otsu



Beras Hitam - Sebelum Thresholding



Beras Hitam - Hasil Otsu



✓ 4.2 Operasi Morfologi

Setelah thresholding, citra biner dapat mengandung noise atau bagian objek yang belum terhubung dengan baik.

Operasi morfologi digunakan untuk memperbaiki hasil segmentasi. Pada tahap ini digunakan operasi **Opening** untuk menghilangkan noise kecil dan **Closing** untuk

membantu menghubungkan bagian objek yang terputus.

```
# OPERASI MORFOLOGI

kernel = np.ones((3, 3), np.uint8)

def morphological_processing(binary_image):
    # Opening: menghilangkan noise kecil
    opening = cv2.morphologyEx(
        binary_image,
        cv2.MORPH_OPEN,
        kernel,
        iterations=1
    )

    # Closing: menutup celah kecil pada objek
    closing = cv2.morphologyEx(
        opening,
        cv2.MORPH_CLOSE,
        kernel,
        iterations=2
    )

    return closing

putih_segment = morphological_processing(putih_thresh)
merah_segment = morphological_processing(merah_thresh)
hitam_segment = morphological_processing(hitam_thresh)

print("Operasi morfologi selesai.")
```

Operasi morfologi selesai.

✓ Visualisasi Hasil Segmentasi

Citra hasil operasi morfologi digunakan sebagai hasil segmentasi awal. Pada hasil ini diharapkan butiran beras dapat terlihat sebagai objek yang terpisah dari background.

```
# VISUALISASI HASIL SEGMENTASI

plt.figure(figsize=(15, 10))

# Beras Putih
plt.subplot(3, 2, 1)
plt.imshow(putih_thresh, cmap="gray")
plt.title("Beras Putih - Sebelum Morfologi")
plt.axis("off")
```

```
plt.subplot(3, 2, 2)
plt.imshow(putih_segment, cmap="gray")
plt.title("Beras Putih - Hasil Segmentasi")
plt.axis("off")

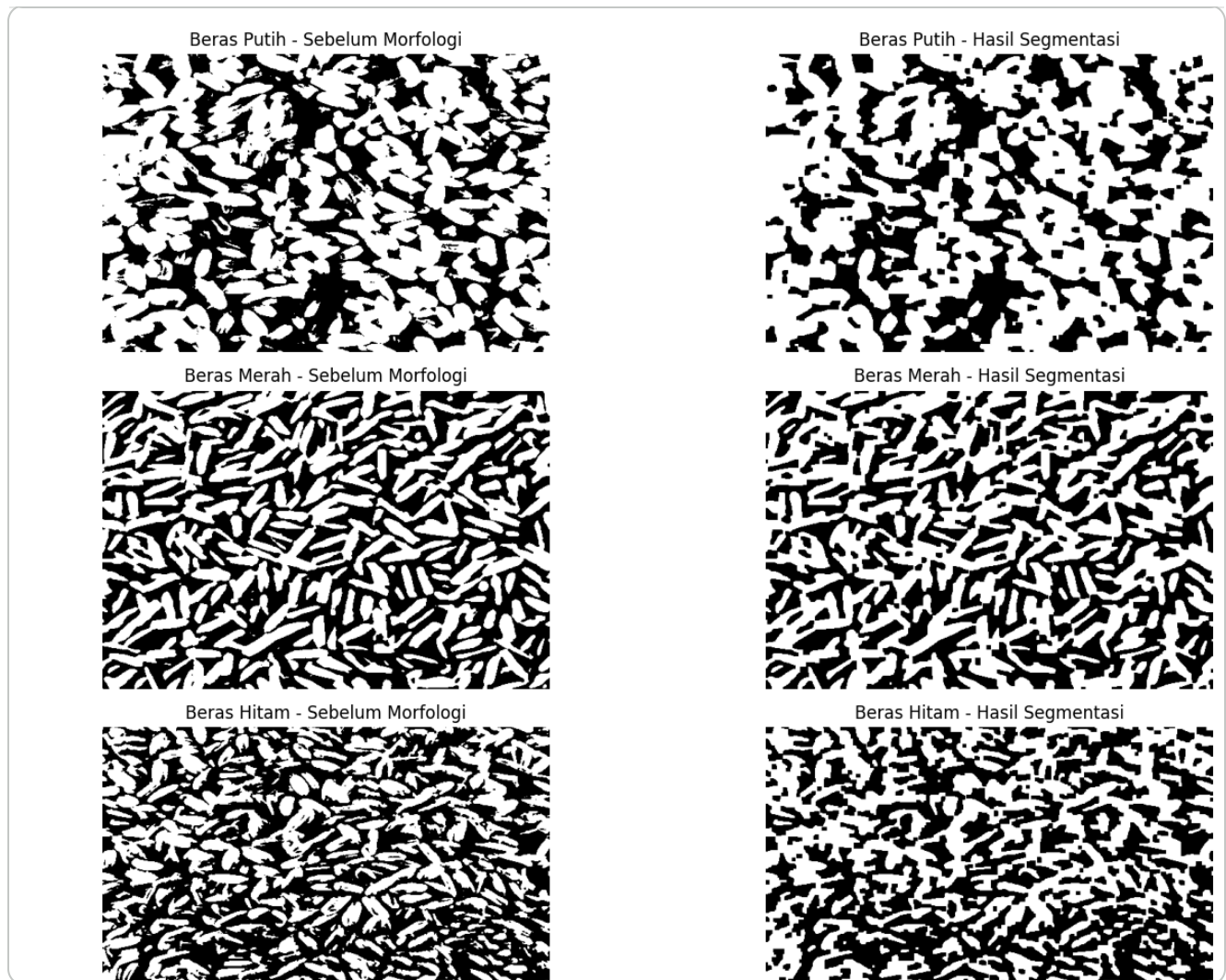
# Beras Merah
plt.subplot(3, 2, 3)
plt.imshow(merah_thresh, cmap="gray")
plt.title("Beras Merah - Sebelum Morfologi")
plt.axis("off")

plt.subplot(3, 2, 4)
plt.imshow(merah_segment, cmap="gray")
plt.title("Beras Merah - Hasil Segmentasi")
plt.axis("off")

# Beras Hitam
plt.subplot(3, 2, 5)
plt.imshow(hitam_thresh, cmap="gray")
plt.title("Beras Hitam - Sebelum Morfologi")
plt.axis("off")

plt.subplot(3, 2, 6)
plt.imshow(hitam_segment, cmap="gray")
plt.title("Beras Hitam - Hasil Segmentasi")
plt.axis("off")

plt.tight_layout()
plt.show()
```



✓ Visualisasi Segmentasi pada Citra Asli

Hasil segmentasi ditampilkan menggunakan mask pada citra asli untuk memastikan bahwa objek yang terdeteksi merupakan butiran beras.

```
# APPLY MASK KE CITRA ASLI

putih_masked = cv2.bitwise_and(
    putih_resize,
    putih_resize,
    mask=putih_segment
)

merah_masked = cv2.bitwise_and(
    merah_resize,
    merah_resize,
    mask=merah_segment
)

hitam_masked = cv2.bitwise_and(
```

```

        hitam_resize,
        hitam_resize,
        mask=hitam_segment
    )

# Visualisasi
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(putih_masked, cv2.COLOR_BGR2RGB))
plt.title("Beras Putih - Hasil Segmentasi")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(merah_masked, cv2.COLOR_BGR2RGB))
plt.title("Beras Merah - Hasil Segmentasi")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(hitam_masked, cv2.COLOR_BGR2RGB))
plt.title("Beras Hitam - Hasil Segmentasi")
plt.axis("off")

plt.tight_layout()
plt.show()

```



✓ 5. Ekstraksi Fitur

Ekstraksi fitur merupakan proses mengambil karakteristik penting dari citra hasil segmentasi untuk direpresentasikan dalam bentuk data numerik.

Pada penelitian ini digunakan fitur warna dan bentuk. Fitur warna diperoleh dari ruang warna RGB dan HSV, sedangkan fitur bentuk diperoleh dari objek hasil segmentasi.

Fitur yang digunakan meliputi:

- Mean Red, Green, dan Blue
- Standard Deviation Red, Green, dan Blue
- Mean Hue, Saturation, dan Value
- Area objek

- Perimeter objek
- Circularity objek

Fitur-fitur tersebut selanjutnya digunakan sebagai masukan pada tahap klasifikasi.

✓ 5.1 Ekstraksi Fitur Warna

Fitur warna digunakan untuk memperoleh karakteristik warna dari masing-masing jenis beras.

Nilai warna dihitung hanya pada area yang teridentifikasi sebagai objek beras berdasarkan hasil segmentasi, sehingga background tidak ikut memengaruhi perhitungan fitur.

```
# EKSTRAKSI FITUR WARNA

def extract_color_features(image, mask):
    # Konversi BGR ke RGB
    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Konversi BGR ke HSV
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # Ambil hanya piksel yang termasuk objek beras
    object_pixels_rgb = rgb[mask > 0]
    object_pixels_hsv = hsv[mask > 0]

    # Mean RGB
    mean_r = np.mean(object_pixels_rgb[:, 0])
    mean_g = np.mean(object_pixels_rgb[:, 1])
    mean_b = np.mean(object_pixels_rgb[:, 2])

    # Standard deviation RGB
    std_r = np.std(object_pixels_rgb[:, 0])
    std_g = np.std(object_pixels_rgb[:, 1])
    std_b = np.std(object_pixels_rgb[:, 2])

    # Mean HSV
    mean_h = np.mean(object_pixels_hsv[:, 0])
    mean_s = np.mean(object_pixels_hsv[:, 1])
    mean_v = np.mean(object_pixels_hsv[:, 2])

    return [
        mean_r, mean_g, mean_b,
        std_r, std_g, std_b,
        mean_h, mean_s, mean_v
    ]
```

```

fitur_putih_warna = extract_color_features(
    putih_resize,
    putih_segment
)

fitur_merah_warna = extract_color_features(
    merah_resize,
    merah_segment
)

fitur_hitam_warna = extract_color_features(
    hitam_resize,
    hitam_segment
)

print("Ekstraksi fitur warna selesai.")

```

Ekstraksi fitur warna selesai.

✓ 5.2 Ekstraksi Fitur Bentuk

Selain warna, karakteristik bentuk objek juga dapat digunakan sebagai fitur.

Fitur bentuk yang digunakan adalah:

- **Area:** luas area objek.
- **Perimeter:** panjang batas atau keliling objek.
- **Circularity:** tingkat kebulatan objek.

Circularity dihitung menggunakan persamaan:

$$\text{Circularity} = 4\pi \times \text{Area} / \text{Perimeter}^2$$

```

# EKSTRAKSI FITUR BENTUK

def extract_shape_features(mask):
    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    # Hilangkan objek yang terlalu kecil
    contours = [
        contour for contour in contours
        if cv2.contourArea(contour) > 20
    ]

    if len(contours) == 0:

```

```

        return [0, 0, 0]

    areas = []
    perimeters = []
    circularities = []

    for contour in contours:
        area = cv2.contourArea(contour)
        perimeter = cv2.arcLength(contour, True)

        if perimeter > 0:
            circularity = (
                4 * np.pi * area /
                (perimeter ** 2)
            )
        else:
            circularity = 0

        areas.append(area)
        perimeters.append(perimeter)
        circularities.append(circularity)

    return [
        np.mean(areas),
        np.mean(perimeters),
        np.mean(circularities)
    ]

fitur_putih_bentuk = extract_shape_features(putih_segment)
fitur_merah_bentuk = extract_shape_features(merah_segment)
fitur_hitam_bentuk = extract_shape_features(hitam_segment)

print("Ekstraksi fitur bentuk selesai.")

```

Ekstraksi fitur bentuk selesai.

✓ 5.3 Membentuk Dataset Fitur

Seluruh fitur warna dan bentuk digabungkan menjadi satu dataset. Setiap citra diberi label sesuai dengan jenis berasnya.

```

# MEMBENTUK DATASET FITUR

nama_fitur = [
    "Mean_R", "Mean_G", "Mean_B",
    "Std_R", "Std_G", "Std_B",
    "Mean_H", "Mean_S", "Mean_V",
    "Mean_Area", "Mean_Perimeter", "Mean_Circularity"
]

```

```

]

data_fitur = [
    fitur_putih_warna + fitur_putih_bentuk,
    fitur_merah_warna + fitur_merah_bentuk,
    fitur_hitam_warna + fitur_hitam_bentuk
]

df_fitur = pd.DataFrame(
    data_fitur,
    columns=nama_fitur
)

df_fitur["Jenis_Beras"] = [
    "Beras Putih",
    "Beras Merah",
    "Beras Hitam"
]

df_fitur

```

	Mean_R	Mean_G	Mean_B	Std_R	Std_G	Std_B	Mean_H
0	209.286636	205.874094	194.182761	9.710342	12.285808	17.135709	27.816557
1	192.069867	107.903237	73.972117	27.807080	28.413390	26.158566	8.582284
2	70.940423	63.790786	61.205407	33.105677	29.468335	28.909703	49.781591

Next steps:

[Generate code with df_fitur](#)[New interactive sheet](#)

Hasil Ekstraksi Fitur

Tabel di atas menunjukkan hasil ekstraksi fitur dalam bentuk numerik. Setiap baris merepresentasikan satu citra jenis beras, sedangkan setiap kolom menunjukkan karakteristik warna atau bentuk yang berhasil diekstraksi.

Data fitur tersebut akan digunakan sebagai dasar untuk proses klasifikasi jenis beras.

6. Klasifikasi

Klasifikasi merupakan tahap untuk menentukan jenis beras berdasarkan fitur yang telah diperoleh pada tahap ekstraksi fitur.

Pada penelitian ini digunakan metode **K-Nearest Neighbors (KNN)**. KNN melakukan klasifikasi berdasarkan kedekatan nilai fitur suatu data terhadap data yang telah

diketahui kelasnya.

Fitur warna dan bentuk yang telah diperoleh digunakan sebagai masukan untuk proses klasifikasi.

✓ 6.1 Persiapan Data Klasifikasi

Sebelum dilakukan klasifikasi, fitur numerik dipisahkan dari label jenis beras. Fitur kemudian dinormalisasi agar setiap fitur memiliki skala yang sebanding.

```
# PERSIAPAN DATA KLASIFIKASI

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# Memisahkan fitur dan label
X = df_fitur.drop(columns=["Jenis_Beras"])
y = df_fitur["Jenis_Beras"]

# Normalisasi fitur
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Data fitur:")
print(X)

print("\nLabel:")
print(y)
```

```
Data fitur:
      Mean_R      Mean_G      Mean_B      Std_R      Std_G      Std_B  \
0  209.286636  205.874094  194.182761   9.710342  12.285808  17.135709
1  192.069867  107.903237   73.972117  27.807080  28.413390  26.158566
2   70.940423   63.790786   61.205407  33.105677  29.468335  28.909703
```

```
      Mean_H      Mean_S      Mean_V      Mean_Area  Mean_Perimeter  \
0  27.816557  19.363271  209.719791  15858.090909      973.181799
1   8.582284  158.905042  192.069867   4541.794118     470.950714
2  49.781591   38.015031   71.184396   1522.727848     271.234448
```

```
      Mean_Circularity
0           0.457394
1           0.402937
2           0.424095
```

```
Label:
0   Beras Putih
1   Beras Merah
2   Beras Hitam
Name: Jenis_Beras, dtype: object
```

✓ 6.2 Membuat Model KNN

Model KNN dibuat dengan menggunakan nilai **K = 1** karena dataset yang tersedia pada penelitian ini terdiri dari satu citra referensi untuk setiap kelas.

Model digunakan untuk mengenali kelas berdasarkan fitur yang memiliki karakteristik paling dekat dengan data masukan.

```
# MODEL KNN

knn = KNeighborsClassifier(n_neighbors=1)

# Melatih model menggunakan data fitur
knn.fit(X_scaled, y)

print("Model KNN berhasil dibuat.")

Model KNN berhasil dibuat.
```

✓ 6.3 Hasil Klasifikasi

Model KNN digunakan untuk melakukan klasifikasi terhadap citra yang digunakan sebagai data referensi.

Hasil prediksi dibandingkan dengan label sebenarnya untuk melihat kesesuaian antara hasil klasifikasi dan jenis beras yang diketahui.

```
# PREDIKSI DATA

y_pred = knn.predict(X_scaled)

hasil_klasifikasi = pd.DataFrame({
    "Jenis Beras Sebenarnya": y,
    "Hasil Prediksi": y_pred
})

hasil_klasifikasi
```

	Jenis Beras Sebenarnya	Hasil Prediksi	
0	Beras Putih	Beras Putih	
1	Beras Merah	Beras Merah	
2	Beras Hitam	Beras Hitam	

